Input/Output

first, let's define names for I/O ecall numbers …

```
.equ SYS_printInt, 244
.equ SYS_readInt, 245
.equ SYS_printChar, 246
.equ SYS_readChar, 247
.equ SYS_printStr, 248
.equ SYS_readStr, 249
.equ SYS_printFloat, 250
.equ SYS_readFloat, 251
```

some examples

to read an integer:

```
li a7, SYS_readInt
ecall           # integer that user types is put into a0
mv s3, a0       # e.g., if want to copy input integer into s3
```

to output an integer:

```
mv a0, s0       # if want to output integer in s0
li a7, SYS_printInt
ecall
```

to output a string:

```
prompt:  .asciz “Enter number\n”        # in .rodata section
# string is represented using ascii binary character code,
# one byte per character,  and zero-terminated

la a0, prompt
li a7, SYS_printStr
ecall
```

to read a string:

```
str1:  .space 128          # in .bss section

la a0, str1
li a1, 128
li a7, SYS_readStr         # will always zero-terminate string
                           # will include newline character if fits
ecall
```

potential complications when read multiple items …

- multiple items (e.g., integers) can be read from the same line of user input
- sometimes awkward, e.g. when reading an integer, if the user types “15” followed by a newline, the newline character is left in the input stream and that is what would be read if the program now tried to read a string

Putting a value into a register

be careful to use the right type of instruction

- lw s4, 0(s1) # content of a memory word put in s4
- la s4, B # address of a memory word put in s4
- li s4, 7
- mv s4, s3

how about putting a single character into register?

prompt: .asciz “Enter number\n” # in .rodata section

la s0, prompt
lbu s1, 3(s0) would put “e” into the low-order byte of register s1, and set other bytes of s1 to 0 (why “e” and not “t”?)

see page 114 in text for ascii character representation

More RISC-V programming examples

(including implementations of control flow constructs, use of arrays and strings)

(reference: text Sections 2.7, 2.9, pages 132-133)

example C code

```
if (A != B) A++;
      else B++;
A = A + A;
```

corresponding RISC-V assembly language code (using s0 for A and s1 for B):

```
        bne s0, s1, incA    # will go to the instruction with label
                            # "incA" if the contents of the two
                            # registers are not the same
        addi s1, s1, 1
        j next              # will go to the instruction with label
                            # "next" (what would happen if we
                            # omitted this instruction?)
incA:   addi s0 ,s0, 1
next:   add s0, s0, s0
```